

Resource Allocation System – Iteration 37

Least-Change Meeting Time Suggestions

Najib Attar

May 2026

1 Iteration Goal

The goal of Iteration 37 is to improve committee meeting time suggestions by ranking valid alternative slots according to the smallest time change from the originally requested slot.

In Iteration 36, the system added common available time suggestions. When a `CommitteeMeetingRequest` was rejected, the system scanned working hours and returned available alternative slots. These suggestions were valid, but they were mainly first-available suggestions and were not ranked by closeness to the requested time.

Iteration 37 improves the suggestion system by applying a least-change policy. The request approval and rejection behavior remains unchanged. This iteration only changes the ordering of suggested alternatives.

2 Changes from Previous Iteration

The main change from Iteration 36 is that suggestions are now ranked before being returned.

The following changes were introduced:

- `MeetingTimeSuggestion` now stores ranking metadata.
- The stored metadata includes `score`, `timeDistanceMinutes`, `dayDistance`, and `conflictCount`.
- `MeetingTimeSuggestionService` now collects all valid candidate slots before ranking.
- Valid slots are scored and sorted using the least-change policy.
- The top 3 ranked suggestions are returned.
- `AllocationService` now outputs `Suggested least-change alternative times`.
- The web Allocation Summary recognizes and displays the new suggestion text.

- The web README was updated to describe least-change suggestions.

Existing rules and allocation strategies remain unchanged.

3 Scenario

A student requests a committee meeting for Monday 10:00-11:00. A required participant is busy from Monday 10:00-12:00, so the meeting cannot be approved at the requested time.

Before this iteration, the system could suggest valid alternative slots, but the suggestions were not ranked by how close they were to the original request. After this iteration, the system ranks the valid alternatives by least change.

For request 41, the ranked suggestions are:

- Monday 09:00-10:00 in space 301, distance 60 minutes.
- Monday 12:00-13:00 in space 301, distance 120 minutes.
- Monday 12:30-13:30 in space 301, distance 150 minutes.

Monday 09:00-10:00 is ranked first because it is only 60 minutes away from the requested start time. This iteration uses absolute time distance, so valid slots before or after the requested time may be suggested.

4 Modules and Their Tasks

Module	Task
CommitteeMeetingRequest	Stores the preferred meeting time, duration, and required participant IDs.
MeetingTimeSuggestion	Stores one suggested slot and its ranking metadata.
MeetingTimeSuggestionService	Generates valid candidates, scores them, sorts them, and returns the best suggestions.
UserAvailabilityService	Checks participant availability using working hours and busy slots.
AllocationService	Triggers suggestion generation after a rejected committee meeting request.
Allocation	Provides existing room allocation data used to avoid room conflicts.
Flask Dashboard	Displays suggestion text exported by the C++ backend.

Table 1: Main modules involved in Iteration 37

5 Least-Change Suggestion Design

The suggestion process still begins by scanning candidate meeting slots. A candidate is valid only if it satisfies the same requirements introduced in Iteration 36:

- it is inside Monday-Friday working days,
- it is inside 09:00-17:00,
- it preserves the original meeting duration,
- all required participants are available,
- the requested room is available.

Invalid candidates are not ranked or returned. This means the ranking process only compares conflict-free suggestions.

The improvement in Iteration 37 is that `MeetingTimeSuggestionService` no longer returns the first valid slots immediately. Instead, it collects all valid candidates, computes a least-change score, sorts the candidates, and returns the best 3 suggestions.

This keeps suggestion logic inside the suggestion service. It does not move ranking into `CommitteeMeetingRequest`, `ParticipantAvailabilityRule`, allocation strategies, or Flask.

6 Scoring Policy

The least-change score is based on the absolute time distance between the originally requested start time and the candidate start time.

$$\text{score} = |(\text{candidateDay} \times 1440 + \text{candidateStartMinutes}) - (\text{requestedDay} \times 1440 + \text{requestedStartMinutes})|$$

The value 1440 is the number of minutes in one day. This converts the day and start time into one comparable minute value.

A smaller score means the candidate requires a smaller change from the originally requested meeting time.

Criterion	Meaning
<code>score</code>	Absolute time distance from requested start time.
<code>timeDistanceMinutes</code>	Difference between requested and candidate start times.
<code>dayDistance</code>	Difference between requested and candidate day.
<code>conflictCount</code>	Reserved for future conflict-aware ranking.

Table 2: Ranking metadata stored in `MeetingTimeSuggestion`

7 Tie-Breaking Rules

If two candidates have the same score, the system applies deterministic tie-breaking rules:

1. Smaller score.
2. Same day preferred.
3. Earlier calendar day.
4. Earlier start time.
5. Lower space ID for deterministic ordering.

These tie-breakers keep the output stable and predictable.

8 Prototype Test Case

Request	Condition	Result
40	All required participants are free	Approved, no suggestions needed
41	Required participant busy Monday 10:00-12:00	Rejected with ranked least-change suggestions

Table 3: Prototype test cases for Iteration 37

For request 41, the requested meeting time is Monday 10:00-11:00. Required participant Ali is busy Monday 10:00-12:00, so the request remains rejected.

The ranked suggestions are:

1. Monday 09:00-10:00 in space 301, distance 60 minutes.
2. Monday 12:00-13:00 in space 301, distance 120 minutes.
3. Monday 12:30-13:30 in space 301, distance 150 minutes.

The first suggestion is before the requested time. This is allowed because the current policy uses absolute time distance. A future-only suggestion mode can be added later if needed.

9 Iteration Comparison

Feature	Before Iteration 37	After Iteration 37
Suggestion ordering	First available valid slots	Valid slots ranked by least-change score
Candidate validity	Required participants and requested room must be available	Same validity rules preserved
Suggestion metadata	Time slot, space ID, and note	Time slot, space ID, note, score, distance, day distance, and conflict count
Request approval/rejection	Unchanged exact-time validation	Still unchanged
Allocation strategies	Not responsible for suggestions	Still not responsible for suggestions
Flask dashboard	Displays exported suggestion text	Displays least-change suggestion text
Alternative room search	Not implemented	Still not implemented
Global optimization	Not implemented	Still not implemented

Table 4: Comparison before and after Iteration 37

10 Updated Class Diagram

The updated class diagram includes `CommitteeMeetingRequest`, `MeetingTimeSuggestion`, `MeetingTimeSuggestionService`, `UserAvailabilityService`, `UserBusySlot`, `AllocationService`, and `TimeSlot`.

It shows that `MeetingTimeSuggestion` stores score and ranking metadata. It also shows that `MeetingTimeSuggestionService` creates and ranks suggestions, uses `UserAvailabilityService`, and is triggered by `AllocationService` after a rejected `CommitteeMeetingRequest`.

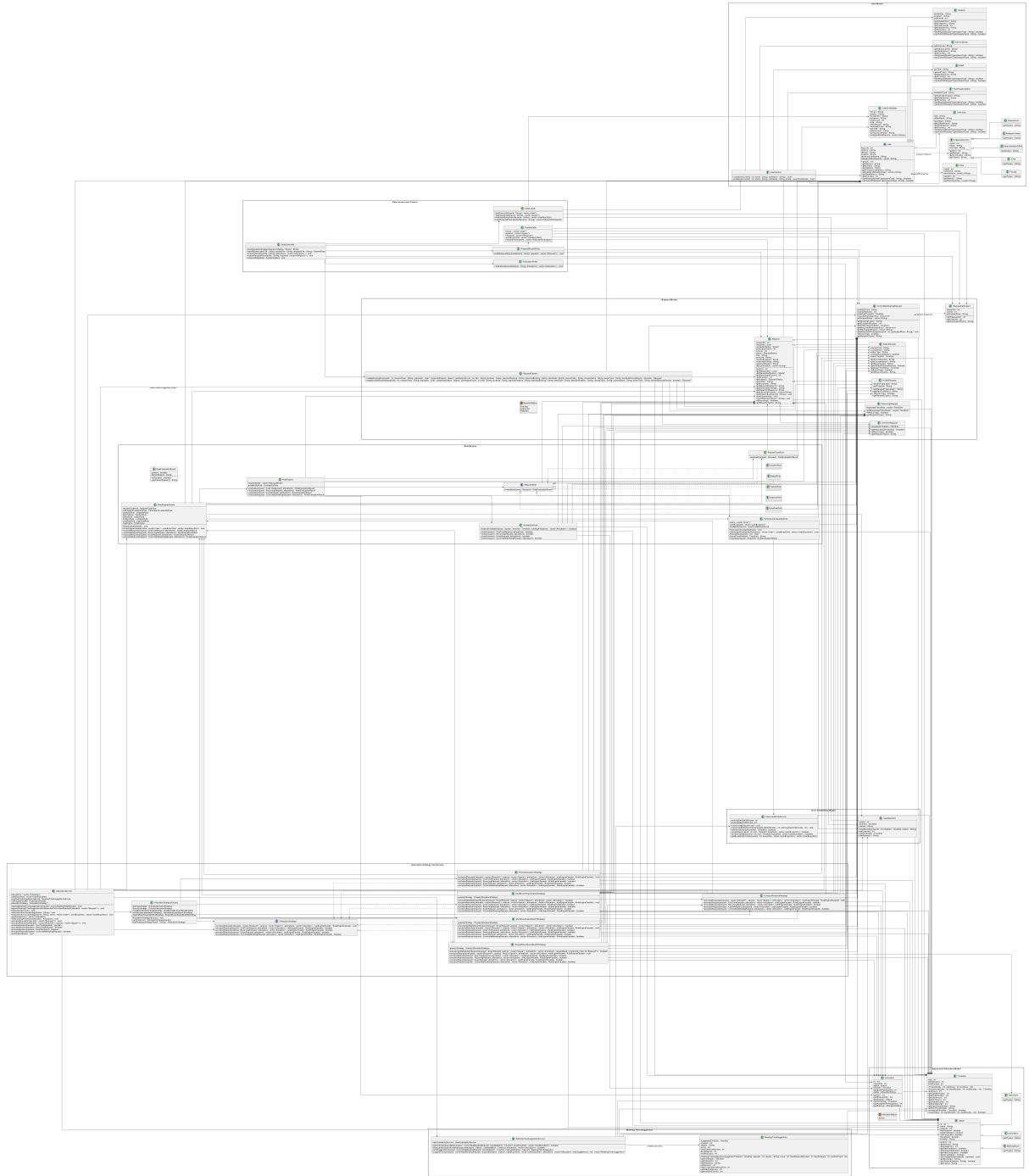


Figure 1: Updated class diagram for Iteration 37

11 Updated Workflow Diagram

The updated workflow shows that `CommitteeMeetingRequest` is processed normally. If the request is rejected, `AllocationService` calls `MeetingTimeSuggestionService`.

The service scans candidate slots, removes invalid candidates, scores valid candidates, sorts them by least-change score, and adds the top 3 suggestions to the request history. The suggestion text is exported through `request_results.csv`, and the web Allocation

Summary displays it.

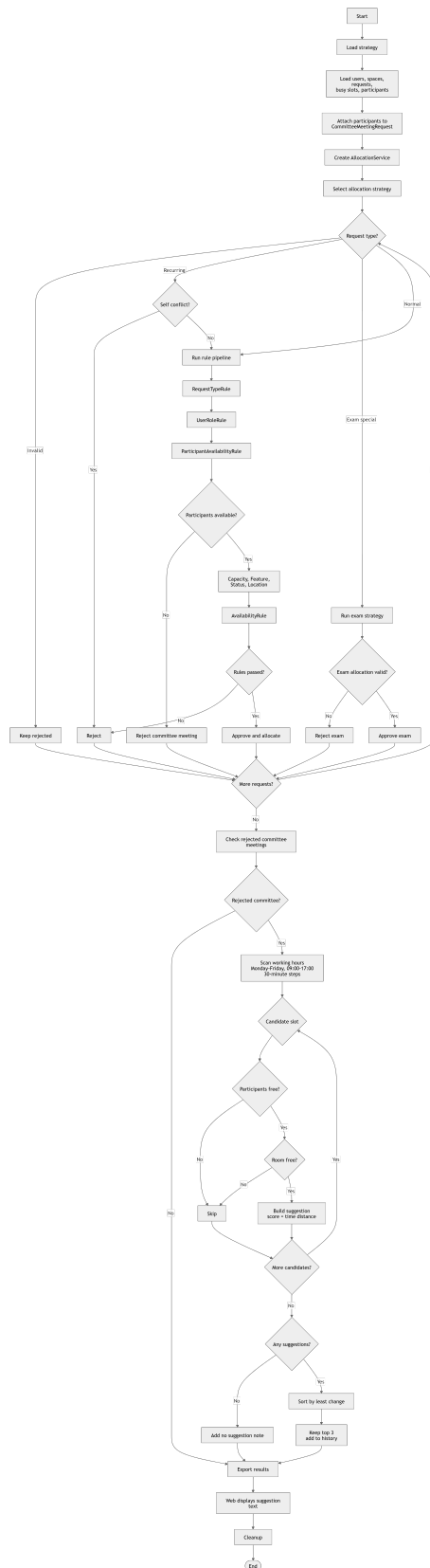


Figure 2: Updated workflow diagram for Iteration 37

12 Prototype Implementation

The prototype implementation confirms that least-change ranking is now applied to valid meeting suggestions.

Request 40 remains approved because all required participants are available. Request 41 remains rejected because a required participant is unavailable at the requested time. The rejection behavior is unchanged, but the request now receives ranked least-change suggestions.

The implementation keeps responsibilities separated. `ParticipantAvailabilityRule` still performs exact-time validation. `AvailabilityRule` still checks physical room availability. Allocation strategies still decide request processing order and allocation behavior. `MeetingTimeSuggestionService` is responsible for ranking suggestions, and Flask only displays backend-generated suggestion text.

13 Current Limitation and Future Work

The current ranking policy uses absolute time distance. Therefore, valid suggestions may be before or after the originally requested time.

The current implementation still uses only the requested room. It does not search for alternative rooms. It also does not rank partially conflicting suggestions because returned suggestions must be fully valid and conflict-free.

Future work may include:

- future-only suggestion mode,
- alternative room search,
- participant conflict ranking,
- least-change ranking with room changes,
- rescheduling support,
- global scheduling optimization.

14 Summary

Iteration 37 improves committee meeting suggestions by ranking valid alternatives using a least-change policy. The system now compares candidate slots by their absolute time distance from the originally requested start time and returns the closest valid options first.

This iteration does not change approval or rejection behavior. It improves the usefulness of suggestion output while preserving the existing architecture. Rules remain responsible for validation, strategies remain responsible for allocation behavior, `AllocationService`

triggers suggestions after rejection, and `MeetingTimeSuggestionService` handles suggestion ranking.